Personal Loan Application System

Database Design Document

Version: 1.0 | Status: In Progress

Property	Value
Database	MySQL 8+
Type	Relational Database (RDBMS)
Storage Engine	InnoDB
Character Set	UTF-8
Collation	utf8mb4_unicode_ci
Total Tables	11 (2 Master + 9 Transaction)

Table of Contents

1. Database Overview

Purpose

The Personal Loan Application System database is designed to manage the complete lifecycle of a loan application. It stores customer information, authentication details, loan applications, uploaded documents, approval history, EMI schedules, notifications, and audit records.

The database follows a normalized relational database design to ensure data consistency, reduce redundancy, and improve maintainability. It also implements banking best practices such as the Maker-Checker approval process, audit logging, and role-based access control (RBAC).

Database Objectives

The database is designed to achieve the following objectives:

- Store customer registration and authentication information.
- Maintain customer profile details.
- Manage multiple loan applications for customers.
- Store loan product information.
- Maintain uploaded customer documents.
- Support Maker-Checker workflow for loan approval.
- Track every loan status change.
- Store EMI calculation details.
- Maintain notification history.
- Record audit logs for important user actions.
- Ensure data integrity using primary keys, foreign keys, and constraints.
- Support future scalability and new loan products.

Database Type

Property	Value
Database Model	Relational Database (RDBMS)
Database Engine	MySQL 8+
Character Set	UTF-8
Storage Engine	InnoDB

Database Design Principles

- Normalized database structure (up to Third Normal Form where appropriate).
- Use of Primary Keys for unique identification.
- Use of Foreign Keys to maintain relationships.
- Elimination of duplicate data.
- Separation of master data and transactional data.
- Soft delete strategy for important records.

- Audit columns in all transactional tables.
- Role-Based Access Control (RBAC).
- Maker-Checker workflow for loan approval.
- Optimized indexing for better query performance.

Master Tables

The following tables store master data:

- roles
- loan_type

These tables contain relatively static data that is referenced by other tables.

Transaction Tables

The following tables store day-to-day business transactions:

- users
- customer_profile
- loan_application
- documents
- approval_history
- loan_status_history
- emi_schedule
- notifications
- audit_log

These tables continuously grow as customers use the application.

Database Modules

Authentication Module

- roles
- users

Customer Module

- customer_profile

Loan Module

- loan_type
- loan_application

Document Management Module

- documents

Approval Module

- approval_history
- loan_status_history

EMI Module

- emi_schedule

Notification Module

- notifications

Audit Module

- audit_log

High-Level Database Flow

```
Customer
  |
  v
User Registration
  |
  v
Customer Profile
  |
  v
Loan Application
  |
  v
Document Upload
  |
  v
Loan Verification
  |
  v
Loan Approval
  |
  v
EMI Generation
  |
  v
Loan Disbursement
  |
  v
Notifications
  |
  v
Audit Logging
```

Total Tables

Category	Count
Master Tables	2
Transaction Tables	9
Total Tables	11

Tables Included

1. roles
2. users
3. customer_profile
4. loan_type
5. loan_application
6. documents
7. approval_history
8. loan_status_history
9. emi_schedule
10. notifications
11. audit_log

Future Enhancements

- Credit Score Integration
- Aadhaar Verification
- PAN Verification
- Payment Gateway
- Loan Foreclosure
- Loan Pre-Closure
- Co-Applicant Management
- Guarantor Management
- Branch Management
- Multi-Tenant Banking Support
- AI-based Loan Eligibility Prediction

2. Database Standards

Purpose

The database standards define the design guidelines and best practices followed while designing the Personal Loan Application System database. These standards ensure consistency, maintainability, scalability, security, and high performance across all database objects.

Database Technology

Property	Value
Database	MySQL 8+
Database Type	Relational Database (RDBMS)
Storage Engine	InnoDB
Character Set	UTF-8
Collation	utf8mb4_unicode_ci

Table Naming Standards

- Use lowercase letters.
- Use snake_case naming convention.
- Table names should be plural where applicable.
- Avoid spaces and special characters.
- Use meaningful names.

Examples

```
roles
users
customer_profile
loan_type
loan_application
documents
approval_history
loan_status_history
emi_schedule
notifications
audit_log
```

Column Naming Standards

- Use lowercase letters.
- Use snake_case.
- Column names should clearly describe the data.

- Avoid abbreviations unless commonly accepted.

Primary Key Standards

- Every table must have a Primary Key.
- Primary Key should be BIGINT AUTO_INCREMENT.
- Primary Key should never change.

Example

```
user_id BIGINT PRIMARY KEY AUTO_INCREMENT
```

Data Type Standards

Data Type	Usage
BIGINT	Primary Keys, Foreign Keys
VARCHAR	Text values
TEXT	Long descriptions
DECIMAL(15,2)	Monetary values
BOOLEAN	True/False values
DATE	Date only
TIMESTAMP	Date and Time
ENUM	Status values

Status Standards

Loan Status

```
DRAFT
SUBMITTED
PENDING
UNDER_VERIFICATION
VERIFIED
APPROVED
REJECTED
SANCTIONED
DISBURSED
CLOSED
```

User Status

```
ACTIVE
INACTIVE
```

```
LOCKED
SUSPENDED
```

Audit Column Standards

Column	Purpose
created_at	Record creation timestamp
updated_at	Last update timestamp
created_by	User who created the record
updated_by	User who last updated the record

Soft Delete Standard

Records should not be permanently deleted. Instead, use:

```
is_deleted BOOLEAN DEFAULT FALSE
```

Benefits:

- Prevent accidental data loss.
- Maintain audit history.
- Support data recovery.
- Preserve business records.

Password Storage Standard

Passwords must never be stored in plain text. Use BCrypt Password Encoder.

```
$2a$10$xxxxxxxxxxxxxxxxxxxxxxxxx
```

Unique Constraint Standards

The following fields should be unique:

- Email
- Mobile Number
- PAN Number
- Aadhaar Number
- Role Code

Indexing Standards

Create indexes on frequently searched columns:

- email

- mobile_number
- loan_number
- application_number
- customer_id
- loan_status

Document Storage Standard

Do not store files directly in the database. Store only:

- File Name
- File Path
- File Type
- File Size

Actual files should be stored in:

- Local Storage (Development)
- AWS S3 / Azure Blob Storage (Production)

Normalization Standard

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)

Security Standards

- Role-Based Access Control (RBAC)
- Encrypted Passwords
- JWT Authentication
- Input Validation
- SQL Injection Prevention
- Audit Logging

Backup & Recovery Standards

- Daily Database Backup
- Weekly Full Backup
- Point-in-Time Recovery (PITR)
- Disaster Recovery Plan

Summary

The database follows enterprise-level design standards to ensure:

- Data Integrity
- Security
- Scalability
- High Performance

- Maintainability
- Consistency
- Auditability
- Banking Compliance

3. Naming Convention

Purpose

The Naming Convention defines the standard rules used for naming database objects in the Personal Loan Application System. Following a consistent naming convention improves readability, maintainability, and collaboration among developers and database administrators.

General Naming Rules

- Use lowercase letters only.
- Use snake_case naming convention.
- Use meaningful and descriptive names.
- Avoid spaces and special characters.
- Avoid database reserved keywords.
- Keep names short but meaningful.
- Maintain consistency across the database.

Primary Key Naming Convention

Rule: Every Primary Key should follow: <table_name>_id

Table	Primary Key
roles	role_id
users	user_id
customer_profile	customer_id
loan_type	loan_type_id
loan_application	loan_id
documents	document_id
approval_history	approval_id
loan_status_history	status_history_id
emi_schedule	emi_schedule_id
notifications	notification_id
audit_log	audit_log_id

Unique Constraint Naming Convention

Rule: uk_<table_name>_<column_name>

```
uk_users_email
uk_users_mobile_number
uk_customer_profile_pan_number
uk_customer_profile_aadhaar_number
uk_roles_role_code
```

Foreign Key Constraint Naming Convention

Rule: fk_<child_table>_<parent_table>

```
fk_users_roles
fk_customer_profile_users
fk_loan_application_customer_profile
fk_documents_loan_application
fk_approval_history_loan_application
fk_emi_schedule_loan_application
```

Index Naming Convention

Rule: idx_<table_name>_<column_name>

```
idx_users_email
idx_users_mobile_number
idx_customer_profile_pan_number
idx_loan_application_status
idx_documents_loan_id
idx_notifications_user_id
```

Trigger Naming Convention

Rule: trg_<table_name>_<action>

```
trg_users_before_insert
trg_loan_application_after_update
```

View Naming Convention

Rule: vw_<business_name>

```
vw_active_customers
vw_pending_loans
vw_monthly_loan_report
```

Stored Procedure Naming Convention

Rule: sp_<business_operation>

```
sp_create_loan_application
sp_approve_loan
```

```
sp_calculate_emi
sp_generate_report
```

Function Naming Convention

Rule: fn_<business_operation>

```
fn_calculate_emi
fn_generate_loan_number
fn_check_eligibility
```

Audit Column Naming Convention

Every transactional table should include:

```
created_at
updated_at
created_by
updated_by
is_deleted
```

Java / Spring Boot Naming Conventions

Layer	Example
Entity	Role, User, CustomerProfile, LoanType, LoanApplication
Repository	RoleRepository, UserRepository, LoanApplicationRepository
Service	RoleService, UserService, LoanApplicationService, DocumentService
API	POST /api/auth/register, GET /api/loans, PUT /api/loans/{loanId}

Summary

Following these naming conventions ensures consistent database design, better readability, easier maintenance, faster development, cleaner SQL queries, enterprise-level coding standards, and improved collaboration among developers.

4. Entity List

Purpose

The Entity List provides an overview of all database entities (tables) used in the Personal Loan Application System. It defines the purpose of each entity, its category, the business module it belongs to, and its primary responsibility within the system.

Entity Summary

Entity Name	Category	Module	Purpose
roles	Master	Authentication	Stores all system roles for RBAC.
users	Transaction	Authentication	Stores user login credentials and authentication details.
customer_profile	Transaction	Customer	Stores customer personal, contact, employment, and identity information.
loan_type	Master	Loan	Stores available loan products such as Personal Loan, Home Loan, Car Loan.
loan_application	Transaction	Loan	Stores loan application details submitted by customers.
documents	Transaction	Document Management	Stores uploaded customer documents and file metadata.
approval_history	Transaction	Loan Approval	Stores approval, rejection, and review history for every loan application.
loan_status_history	Transaction	Loan Tracking	Maintains the complete history of loan status changes.
emi_schedule	Transaction	EMI Management	Stores EMI calculation details and repayment information.
notifications	Transaction	Notification	Stores email, SMS, and in-app notification history.
audit_log	Transaction	Audit	Stores audit records of important user activities.

Total Entities

Category	Count
Master Entities	2
Transaction Entities	9
Total Entities	11

Module-wise Entity Mapping

Module	Entities
Authentication	roles, users
Customer Management	customer_profile
Loan Management	loan_type, loan_application
Document Management	documents
Loan Approval	approval_history, loan_status_history
EMI Management	emi_schedule
Notification Management	notifications
Audit & Monitoring	audit_log

Entity Dependencies

```
roles
  |
  v
users
  |
  v
customer_profile
  |
  v
loan_application
  |-- loan_type
  |-- documents
  |-- approval_history
  |-- loan_status_history
  |-- emi_schedule
  |-- notifications
  |
  v
audit_log
```

5. Relationship Matrix

Purpose

The Relationship Matrix defines how entities (tables) are related within the Personal Loan Application System. It describes the relationship type, parent table, child table, foreign key, and the business reason behind each relationship.

Relationship Summary

Parent Table	Child Table	Relationship	Foreign Key	Business Reason
roles	users	One-to-Many (1:N)	users.role_id	One role assigned to multiple users.
users	customer_profile	One-to-One (1:1)	customer_profile.user_id	Every customer has one profile.
customer_profile	loan_application	One-to-Many (1:N)	loan_application.customer_id	One customer can submit multiple loan applications.
loan_type	loan_application	One-to-Many (1:N)	loan_application.loan_type_id	One loan type selected by many customers.
loan_application	documents	One-to-Many (1:N)	documents.loan_id	One loan application requires multiple documents.
loan_application	approval_history	One-to-Many (1:N)	approval_history.loan_id	Every approval or rejection is recorded separately.
loan_application	loan_status_history	One-to-Many (1:N)	loan_status_history.loan_id	Every status change is stored for auditing.
loan_application	emi_schedule	One-to-One (1:1)	emi_schedule.loan_id	One approved

Parent Table	Child Table	Relationship	Foreign Key	Business Reason
				loan has one EMI schedule.
users	notifications	One-to-Many (1:N)	notifications.user_id	A user can receive multiple notifications.
users	audit_log	One-to-Many (1:N)	audit_log.user_id	Every user action is recorded for auditing.

Cascade Strategy

Relationship	ON DELETE	ON UPDATE
roles -> users	RESTRICT	CASCADE
users -> customer_profile	CASCADE	CASCADE
customer_profile -> loan_application	RESTRICT	CASCADE
loan_application -> documents	CASCADE	CASCADE
loan_application -> approval_history	RESTRICT	CASCADE
loan_application -> loan_status_history	RESTRICT	CASCADE
loan_application -> emi_schedule	CASCADE	CASCADE
users -> notifications	CASCADE	CASCADE
users -> audit_log	RESTRICT	CASCADE

Interview Questions

Q1. Why is Customer Profile to Loan Application One-to-Many?

Because one customer can submit multiple loan applications over time.

Q2. Why is Approval History a separate table?

To maintain the complete approval and rejection history instead of overwriting previous decisions.

Q3. Why is Loan Status History separated from Loan Application?

To preserve every status transition for auditing and tracking purposes.

Q4. Why is Loan Application to EMI Schedule One-to-One?

Because one approved loan generates one EMI schedule in our current business design.

Q5. Why do we use Foreign Keys?

Foreign Keys enforce referential integrity, prevent orphan records, and maintain valid relationships between tables.

6. Entity Relationship Diagram (ER Diagram)

Purpose

The Entity Relationship Diagram (ER Diagram) provides a high-level visual representation of the database structure for the Personal Loan Application System. It illustrates how different entities (tables) are connected through primary keys and foreign keys.

ER Diagram

Relationship Summary

Parent Entity	Child Entity	Relationship
roles	users	One-to-Many
users	customer_profile	One-to-One
customer_profile	loan_application	One-to-Many
loan_type	loan_application	One-to-Many
loan_application	documents	One-to-Many

Parent Entity	Child Entity	Relationship
loan_application	approval_history	One-to-Many
loan_application	loan_status_history	One-to-Many
loan_application	emi_schedule	One-to-One
users	notifications	One-to-Many
users	audit_log	One-to-Many

Loan Status Flow

```
DRAFT
  |
  v
SUBMITTED
  |
  v
PENDING
  |
  v
UNDER_VERIFICATION
  |
  v
APPROVED
  |
  v
DISBURSED
```

Design Principles Followed

- Master and transaction tables are separated.
- Relationships are enforced using Foreign Keys.
- One-to-One, One-to-Many, and Many-to-One relationships are used where appropriate.
- Business history is maintained using separate history tables.
- Approval workflow follows the Maker-Checker principle.
- Database design follows normalization principles to reduce redundancy and improve maintainability.

Benefits of the ER Diagram

- Provides a clear understanding of entity relationships.
- Simplifies database implementation.
- Helps developers write efficient SQL queries.
- Supports future database enhancements.
- Improves communication between developers, architects, and business analysts.
- Serves as the foundation for database creation and API design.

Next Phase - Table Design Specification

After completing the ER Diagram, the next phase is the Table Design Specification. The following tables will be designed in detail:

12. roles
13. users
14. customer_profile
15. loan_type
16. loan_application
17. documents
18. approval_history
19. loan_status_history
20. emi_schedule
21. notifications
22. audit_log

Each table will include:

- Purpose
- Business Rules
- Column Details
- Data Types
- Constraints
- Primary Keys
- Foreign Keys
- Indexes
- Sample Data
- Interview Questions